CSS កម្រិតខ្ពស់

ឯកសារនេះបង្ហាញអំពីបច្ចេកទេស CSS កម្រិតខ្ពស់ ដែលចាំបាច់សម្រាប់អ្នកអភិវឌ្ឍន៍វេប និងអ្នករចនាផ្នែកមុខ។ ចាប់ពី Flexbox និង CSS Grid រហូតដល់ Transformation, Transition, និង Media Query – ឯកសារនេះគ្របដណ្តប់ឧបករណ៍ CSS ទំនើបទាំងអស់ ដែលអ្នកត្រូវការដើម្បីបង្កើត layout ស្អាត responsive និងមានភាពជីវចល។

តារាងមាតិកា

01

Flexbox

CSS Layout Module សម្រាប់រៀបចំ elements ក្នុង row ឬ column

02

CSS Grid

2D Layout System សម្រាប់គ្រប់គ្រង rows និង columns ក្នុងពេលតែមួយ

03

Transformation

2D និង 3D Transform Functions សម្រាប់ផ្លាស់ទី បង្វិល និងប្តូរទំហំ

04

Transition

ការផ្លាស់ប្តូរ style property ជាបន្តិចៗក្នុងរយៈពេលកំណត់

05

Media Query

បច្ចេកទេសសម្រាប់ Responsive Web Design លើគ្រប់ screen size

ជំពូក ១

Flexbox

Flexbox គឺជាអ្វី?

Flexbox គឺជា **CSS Layout Module** ដែលប្រើសម្រាប់រៀបចំ elements នៅក្នុង container មួយឲ្យមាន **flexible និង responsive**។ វាអាចរៀបចំ **row ឬ column**, កំណត់ **alignment**, និងចែក **space** រវាង elements បានយ៉ាងងាយស្រួល។

ហេតុអ្វីប្រើ Flexbox?

Flexbox ត្រូវបានប្រើយ៉ាងទូលំទូលាយក្នុង **modern web design** ដើម្បីបង្កើត layout ដែលអាចប្តូរតាម screen size។ វាដោះស្រាយបញ្ហា alignment ដ៏ស្មុគ្រស្មាញ ដែលពីមុនត្រូវប្រើ float ឬ positioning។

- ស្ថាបត្យកម្ម layout ដ៏សាមញ្ញ
- Responsive ដោយស្វ័យប្រវត្តិ
- គ្រប់គ្រង alignment ងាយស្រួល

Flexbox ចាប់ផ្តើមដោយកំណត់ `display: flex` នៅលើ container element បន្ទាប់មក properties ផ្សេងទៀតត្រូវបានប្រើដើម្បីគ្រប់គ្រងទិសដៅ alignment និង spacing។

FLEX CONTAINER

១.១. Flex Container Properties

Flex Container Properties ត្រូវបានអនុវត្តនៅលើ **parent element** ដែលមាន `display: flex` ។ Properties ទាំងនេះគ្រប់គ្រងរបៀបដែល flex items ត្រូវបានរៀបចំនៅក្នុង container។

Property	តម្លៃ	អត្ថន័យ
display	flex | inline-flex	ចាប់ផ្តើម Flexbox context
flex-direction	row | row-reverse | column | column-reverse	កំណត់ទិសដៅ main axis
flex-wrap	nowrap | wrap | wrap-reverse	គ្រប់គ្រងការ wrap items
flex-flow	direction wrap	Shorthand សម្រាប់ direction + wrap
justify-content	flex-start | center | flex-end | space-between | space-around | space-evenly	Align items លើ main axis
align-items	stretch | flex-start | center | flex-end | baseline	Align items លើ cross axis
align-content	flex-start | center | flex-end | space-between | stretch	Align multiple lines
gap	value	ចន្លោះរវាង flex items

FLEX ITEMS

១.២. Flex Item Properties

Flex Item Properties ត្រូវបានអនុវត្តនៅលើ **child elements** នៅក្នុង flex container។ Properties ទាំងនេះគ្រប់គ្រងរបៀបដែល item ផ្ទាល់ខ្លួនប្រព្រឹត្ត និងបង្ហាញខ្លួន។

Property	តម្លៃ	អត្ថន័យ
order	number (default: 0)	គ្រប់គ្រងលំដាប់ items
flex-grow	number (default: 0)	ការពង្រីក item ដើម្បីបំពេញ space
flex-shrink	number (default: 1)	ការបង្រួម item ពេល space តូច
flex-basis	auto | value	ទំហំដំបូង (initial size) នៃ item
flex	grow shrink basis	Shorthand សម្រាប់ grow, shrink, basis
align-self	auto | flex-start | center | flex-end | stretch	Override align-items សម្រាប់ item តែមួយ

គន្លឹះ៖ ប្រើ `flex: 1` ដើម្បីឲ្យ item ពង្រីកដោយស្មើគ្នា។ ប្រើ `align-self: center` ដើម្បី center item តែមួយដោយមិនប៉ះពាល់ items ផ្សេង។

ជំពូក ២

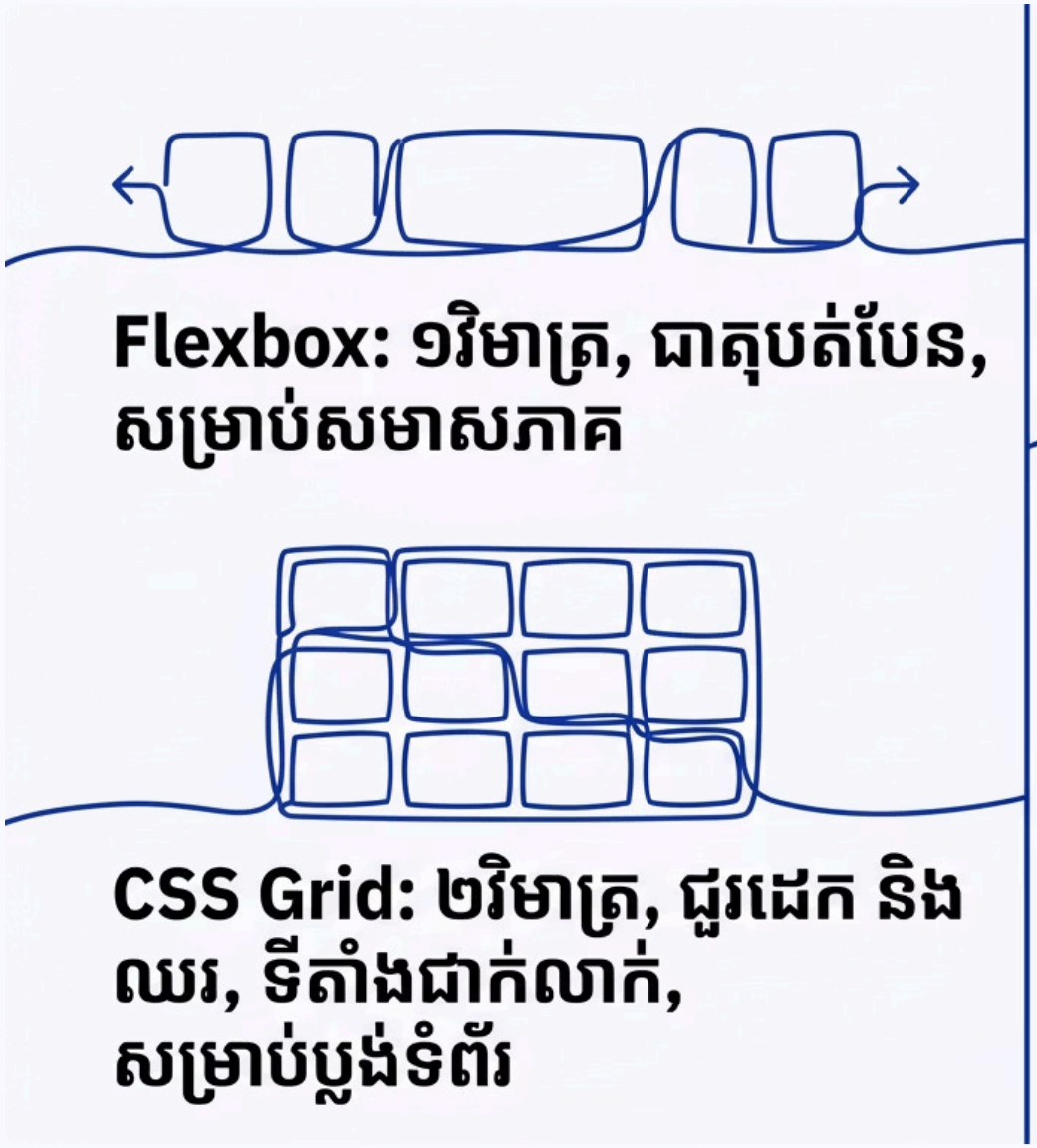
CSS Grid

CSS Grid គឺជាអ្វី?

CSS Grid គឺជា **CSS Layout System** ដែលប្រើសម្រាប់រៀបចំ elements នៅក្នុង **row និង column** ក្នុងពេលតែមួយ។ វាជា **2-Dimensional Layout** ដែលអាចគ្រប់គ្រង layout បានលម្អិត និងមានប្រសិទ្ធភាពសម្រាប់ការបង្កើត web page ឬ complex layout។

លក្ខណៈពិសេស

- រៀបចំ layout ជា **grid (rows + columns)**
- កំណត់ទីតាំង element បានច្បាស់
- ងាយស្រួលសម្រាប់ **responsive design**
- គ្រប់គ្រង alignment លើ 2 axis ក្នុងពេលតែមួយ



GRID CONTAINER

២.១. Grid Container Properties

Grid Container Properties ត្រូវបានអនុវត្តនៅលើ **parent element** ដែលមាន `display: grid` ។ Properties ទាំងនេះគ្រប់គ្រងរចនាសម្ព័ន្ធ grid ទាំងមូល រួមមាន rows, columns, gap, និង alignment។

Property	ឧទាហរណ៍	អត្ថន័យ
`display`	`grid | inline-grid`	ចាប់ផ្តើម Grid context
`grid-template-columns`	`1fr 2fr | repeat(3,1fr)`	កំណត់ columns
`grid-template-rows`	`100px auto`	កំណត់ rows
`grid-template-areas`	`"header header" "sidebar main"`	Named areas សម្រាប់ layout
`gap / row-gap / column-gap`	`16px`	ចន្លោះរវាង grid cells
`justify-items`	`start | center | end | stretch`	Align items horizontally
`align-items`	`start | center | end | stretch`	Align items vertically
`place-items`	`center`	Shorthand justify + align items
`justify-content`	`space-between`	Align grid horizontally
`align-content`	`center`	Align grid vertically

GRID TRACKS

ក. CSS Grid Tracks (Rows and Columns)

Grid Tracks គឺជា rows និង columns ដែលបង្កើតឡើងដោយ grid lines។ យើងកំណត់ tracks ដោយប្រើ `grid-template-columns` និង `grid-template-rows` ។

Columns

```
.container {
  display: grid;
  grid-template-columns:
    200px 1fr 2fr;
}
```

បង្កើត 3 columns: ១ fixed, ២ flexible

Rows

```
.container {
  display: grid;
  grid-template-rows:
    100px auto 50px;
}
```

បង្កើត 3 rows: header, content, footer

Grid tracks អាចប្រើ units ផ្សេងៗ ដូចជា `px`, `%`, `em`, `auto`, និង `fr` (fractional unit)។ `fr` គឺជា unit ពិសេសសម្រាប់ Grid ដែលចែក space ដែលនៅសល់ជា fractions។

GRID GAP

ខ. Grid Gap

Grid Gap កំណត់ចន្លោះ (gutter) រវាង rows និង columns ក្នុង grid។ Property នេះជួយឲ្យ layout មើលទំរង់ស្អាត ដោយមិនចាំបាច់ប្រើ margin ។

Property	ឧទាហរណ៍	អត្ថន័យ
`gap`	`gap: 16px`	Shorthand សម្រាប់ row + column gap
`row-gap`	`row-gap: 20px`	ចន្លោះរវាង rows
`column-gap`	`column-gap: 10px`	ចន្លោះរវាង columns

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px 10px; /* row-gap column-gap */
}
```

ចំណាំ៖ `gap` ជំនួស `grid-gap` ចាស់។ Browser ទំនើបទាំងអស់គាំទ្រ `gap` ។ Property នេះក៏ប្រើបានជាមួយ Flexbox ផងដែរ។

GRID ALIGN

គ. Grid Alignment

CSS Grid ផ្តល់ properties សម្រាប់ **alignment** ទាំងក្នុង container និង items។ Alignment ត្រូវបានគ្រប់គ្រងតាម **inline axis (horizontal)** និង **block axis (vertical)**។

justify-items

Align **items** horizontally នៅក្នុង cell

```
justify-items:
center;
```

align-items

Align **items** vertically នៅក្នុង cell

```
align-items: start;
```

justify-content

Align **grid** horizontally ក្នុង container

```
justify-content:
space-between;
```

align-content

Align **grid** vertically ក្នុង container

```
align-content:
center;
```

REPEAT()

ឃ. មុខងារ repeat()

មុខងារ repeat() ជួយសម្រួលការសរសេរ grid tracks ច្រើន ដែលមានទំហំដូចគ្នា។ វាធ្វើឲ្យ code កាន់តែខ្លី ច្បាស់លាស់ និងងាយថែទាំ។

Syntax

```
repeat(count, size)
```

ដែល **count** គឺចំនួន tracks និង **size** គឺទំហំនៃ track នីមួយៗ

ឧទាហរណ៍

```
/* 3 columns of equal width */
grid-template-columns: repeat(3, 1fr);

/* auto-fill columns */
grid-template-columns:
  repeat(auto-fill, minmax(200px, 1fr));
```

តម្លៃ auto-fill និង auto-fit ក្នុង repeat() ជាមួយ minmax() គឺជា combination ដ៏មានឥទ្ធិពលបំផុតសម្រាប់បង្កើត responsive grid ដោយស្វ័យប្រវត្តិ ដោយមិនចាំបាច់ Media Query។

FR UNIT

ង. ខ្នាត FR (Fractional Unit)

ខ្នាត `fr` គឺជា unit ពិសេសសម្រាប់ CSS Grid ដែលតំណាងឲ្យ **ភាគមួយ (fraction)** នៃ space ដែលនៅសល់ក្នុង grid container។ វាធ្វើឲ្យ columns ឬ rows ចែក space ដោយស្វ័យប្រវត្តិ និងឆ្លើយតបដោយ responsive។

1fr 1fr 1fr

Space ចែកស្មើៗ ៣ ផ្នែក (33.33% នីមួយៗ)

```
repeat(3, 1fr)
```

1fr 2fr

Column ទីពីរធំជាង ២ ដង

1fr 2fr

200px 1fr

Sidebar fixed, main content flexible

200px 1fr

`fr` គណនា space *បន្ទាប់ពី* fixed sizes ត្រូវបានដក។ ដូច្នេះ `200px 1fr 2fr` ឆ្លៀតយក ২০০px ក្នុង column ដំបូង ហើយ space ដែលនៅសល់ត្រូវបានចែក 1:2 ។

GRID ITEMS

២.២. Grid Item Properties

Grid Item Properties ត្រូវបានអនុវត្តនៅលើ **child elements** ក្នុង grid container។ Properties ទាំងនេះគ្រប់គ្រងទីតាំង ទំហំ និង alignment នៃ item ម្នាក់ៗ។

Property	ឧទាហរណ៍	អត្ថន័យ
`grid-column-start`	`1`	Column line ចាប់ផ្តើម
`grid-column-end`	`3`	Column line បញ្ចប់
`grid-column`	`1 / 3`	Shorthand: start / end column
`grid-row-start`	`1`	Row line ចាប់ផ្តើម
`grid-row-end`	`2`	Row line បញ្ចប់
`grid-row`	`1 / 2`	Shorthand: start / end row
`grid-area`	`header`	Named area ឬ shorthand row/column
`justify-self`	`center`	Horizontal align item ១
`align-self`	`end`	Vertical align item ១

NAMED AREAS

ក. Naming Grid Items

CSS Grid អនុញ្ញាតឲ្យ **ដាក់ឈ្មោះ** (name) ដល់ grid areas ដើម្បីឲ្យ layout ងាយស្រួលយល់ និងច្បាស់លាស់ជាងការប្រើ line numbers។ Property `grid-template-areas` ក្នុង container កំណត់ layout structure ដោយប្រើ string។

Container (Named Areas)

```
.container {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 200px 1fr;
}
```

Items (ប្រើ Area Name)

```
header { grid-area: header; }
.sidebar { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }
```

Item ម្នាក់ៗប្រើ `grid-area` ដើម្បីចូលទៅ area ដែលបានដាក់ឈ្មោះ

ការប្រើ named areas ធ្វើឲ្យ code ងាយស្រួលអាន ជាពិសេសសម្រាប់ complex page layouts។ ដើម្បីបញ្ជាក់ empty cell ប្រើ `.` (dot) ក្នុង template area string។

ALIGNMENT

ខ. Grid Items – Alignment

Grid items អាចត្រូវបាន align ដោយ individual ដោយប្រើ `justify-self` (horizontal) និង `align-self` (vertical)។ Properties ទាំងនេះ **override** container-level alignment សម្រាប់ item ជាក់លាក់តែមួយ។

justify-self

គ្រប់គ្រង horizontal alignment នៃ item ១ ក្នុង cell របស់វា

```
.item { justify-self: center; }
```

align-self

គ្រប់គ្រង vertical alignment នៃ item ១ ក្នុង cell របស់វា

```
.item { align-self: end; }
```

place-self

Shorthand សម្រាប់ align-self + justify-self ក្នុងតែ property ១

```
.item { place-self: center; }
```

តម្លៃ	ឥរិយាបទ
`start`	Align ឆ្ពោះទៅ ដើម cell
`end`	Align ឆ្ពោះទៅ ចុង cell
`center`	Center នៅក្នុង cell
`stretch`	ពង្រីកទូទាំង cell (default)

ORDER

គ. Grid Items – Order

CSS Grid អនុញ្ញាតឲ្យ **ផ្លាស់ប្តូរលំដាប់** (order) នៃ grid items ដោយស្ថិតស្ថេរ ដោយប្រើ `order` property ឬ ដោយ redefine placement ជាមួយ `grid-column` និង `grid-row`។

ប្រើ order Property

```
.item-1 { order: 3; }
.item-2 { order: 1; }
.item-3 { order: 2; }
```

Default order គឺ `0` – items ដែលមាន order ខ្ពស់ជាង ត្រូវបានដាក់ក្រោយ

ប្រើ grid-column / grid-row

```
/* Item ចូលទៅ column 2, row 1 */
.item-special {
  grid-column: 2;
  grid-row: 1;
}
```

វិធីនេះគ្រប់គ្រងទីតាំង item បានច្បាស់ជាង

ប្រយ័ត្ន៖ ការប្រើ `order` ផ្លាស់ប្តូរ visual order តែប៉ុណ្ណោះ មិនផ្លាស់ប្តូរ DOM order ទេ។ នេះអាចប៉ះពាល់ដល់ **accessibility** និង keyboard navigation ។ ប្រើដោយប្រុងប្រយ័ត្ន!

ជំពូក ៣

Transformation

CSS **Transformation** ជាបច្ចេកទេសសម្រាប់ **ផ្លាស់ប្តូររូបរាង** (shape), **ទំហំ** (size), **ទីតាំង** (position), និង **ទំនោរ** (angle) នៃ elements ដោយមិនប៉ះពាល់ flow នៃ document ។ CSS Transform ចែកជា **2D** និង **3D** Transform ដែលផ្តល់ល្បឿន animation និង visual effects ដ៏ទាក់ទាញ។

2D Transform

ផ្លាស់ប្តូរ element នៅលើ **X និង Y axis** (horizontal និង vertical)

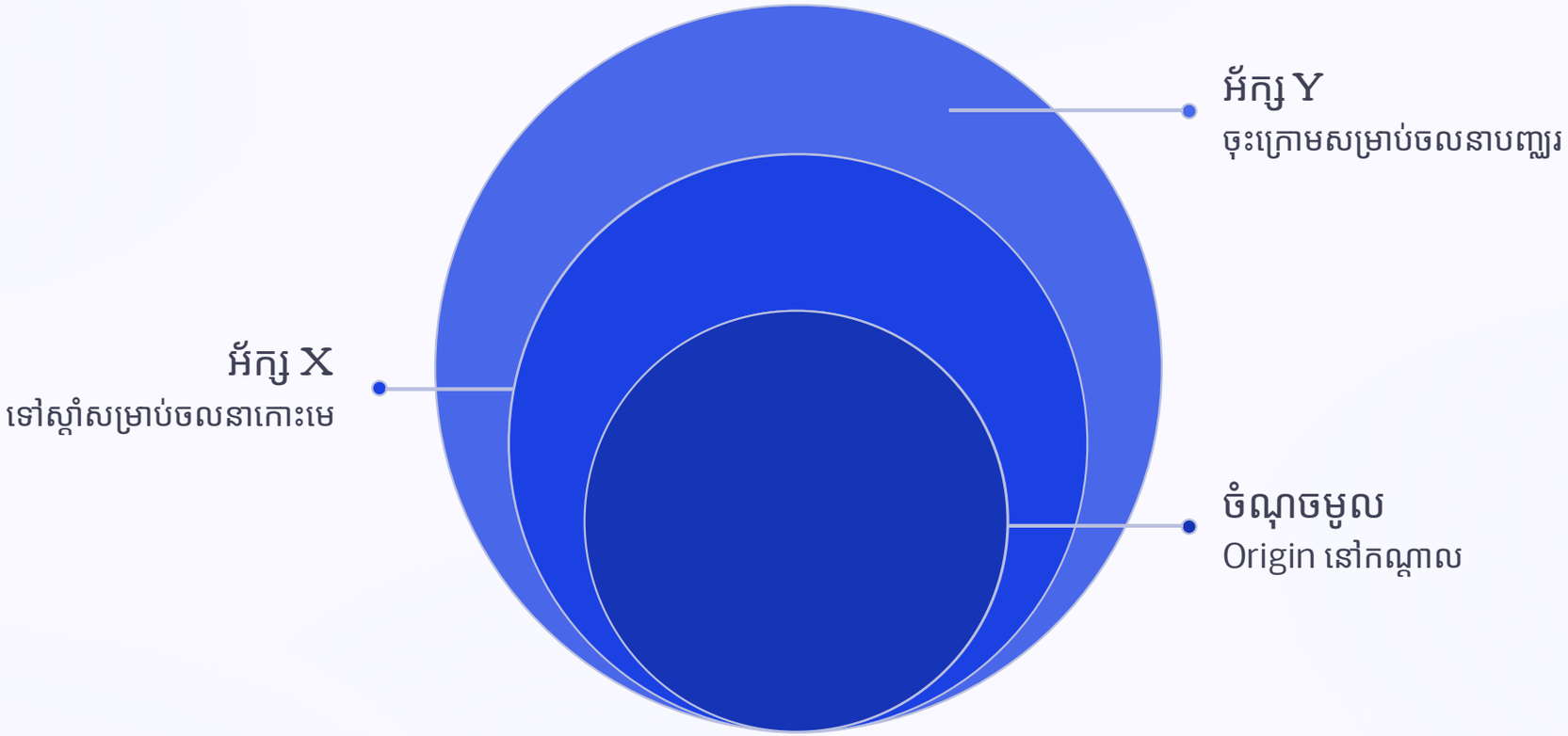
3D Transform

ផ្លាស់ប្តូរ element នៅលើ **X, Y, និង Z axis** (including depth)

2D TRANSFORM

2D Transform គឺជាអ្វី?

2D Transform អនុញ្ញាតឲ្យ element **ផ្លាស់ទី** (translate), **បង្វិល** (rotate), ឬ **ប្តូរទំហំ** (scale) នៅលើ **X និង Y axis**។ Transform ទាំងនេះអនុវត្តតាម Property `transform` ហើយអាច **chain** ច្រើន functions ក្នុងតែ property ១ ។



CSS Transform ប្រើ coordinate system ដែលចាប់ផ្តើមពី **transform-origin** (default: center)។ X axis ដើរពីឆ្វេងទៅស្តាំ, Y axis ដើរពីលើទៅក្រោម។ លក្ខណៈពិសេស transform ស្ថិតនៅក្រៅ document flow ដូច្នេះ elements ផ្សេង មិនត្រូវបាន affect ឡើយ។

2D FUNCTIONS

2D Transform Functions

translate()

ផ្លាស់ទី element ទៅ X, Y position ថ្មី

```
transform: translate(50px, 100px);
```

rotate()

បង្វិល element ជា degrees (deg)

```
transform: rotate(45deg);
```

scale()

ពង្រីក ឬបង្រួម element ជា ratio

```
transform: scale(1.5);
```

skew()

ទំនោរ (tilt) element ជា degrees

```
transform: skew(20deg, 10deg);
```

ឧទាហរណ៍

2D Transform – Code Example

ឧទាហរណ៍ខាងក្រោមបង្ហាញការ **chain** ច្រើន transform functions ក្នុងតែ property ១ ដើម្បីបង្កើត hover effect ដ៏ទាក់ទាញ។

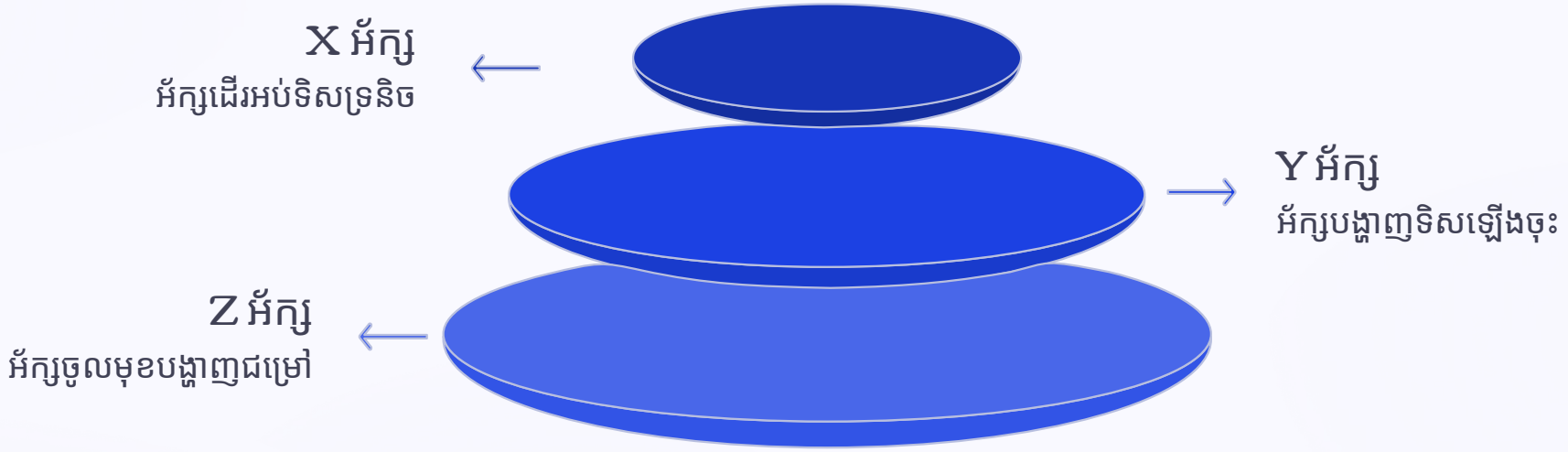
```
/* 2D Transform Example */
.box {
  width: 100px;
  height: 100px;
  background-color: #4967E9;
  transition: transform 0.3s ease;
}

.box:hover {
  transform: scale(1.2) rotate(10deg);
}
```

ការ chain Transform: ពេល chain ច្រើន transform functions, លំដាប់មានសារសំខាន់ណាស់។ `translate(50px) rotate(45deg)` ខុសពី `rotate(45deg) translate(50px)` ព្រោះ rotation ផ្លាស់ប្តូរ coordinate axis ។

3D TRANSFORM

3D Transform គឺជាអ្វី?

3D Transform ពង្រីក 2D Transform ដោយបន្ថែម **Z axis** ដែលតំណាងឲ្យ **depth** (ជម្រៅ) ។ វាអនុញ្ញាតឲ្យ elements ម្លឹងចូលឆ្ពោះអ្នកប្រើ ឬ ចូលទៅក្នុង screen ដើម្បីបង្កើត **3D perspective effect** ដ៏ស្រស់ស្អាត។



ដើម្បីឲ្យ 3D Transform ដំណើរការបានល្អ ត្រូវប្រើ `perspective` property នៅលើ parent element ដើម្បីកំណត់ ចំណុចបង្ហាញ (vanishing point)។ តម្លៃ `perspective` តូចជាង → effect 3D ខ្លាំងជាង។

3D FUNCTIONS

3D Transform Functions

rotateX()

បង្វិល element ជុំវិញ X axis (somersault)

rotateY()

បង្វិល element ជុំវិញ Y axis (spin)

rotateZ()

បង្វិល element ជុំវិញ Z axis (2D rotation)

translateZ()

ផ្លាស់ទី element ឆ្ពោះ ឬ ចេញពី viewer

perspective()

កំណត់ perspective ដើម្បី depth effect

ឧទាហរណ៍

3D Transform – Code Example

ឧទាហរណ៍ **Card Flip Effect** ដ៏ពេញនិយម ដែលប្រើ `rotateY(180deg)` ។

```
/* 3D Card Flip Effect */
.card-container {
  perspective: 1000px; /* set perspective on parent */
}

.card {
  width: 200px;
  height: 150px;
  transition: transform 0.6s ease;
  transform-style: preserve-3d;
}

.card:hover {
  transform: rotateY(180deg);
}

/* Front and Back faces */
.card-front,
.card-back {
  position: absolute;
  backface-visibility: hidden;
}

.card-back {
  transform: rotateY(180deg);
}
```

គន្លឹះ: Property `transform-style: preserve-3d` ចាំបាច់ណាស់ ដើម្បីឲ្យ child elements ថែរក្សា 3D context ។ `backface-visibility: hidden` លាក់ face ខាងក្រោយ ពេលបង្វិល 180 degrees។

ជំពូក ៤

CSS Transition

CSS Transition គឺជាវិធីធ្វើឲ្យ **style property** ផ្លាស់ប្តូរពីតម្លៃមួយទៅមួយផ្សេងទៀត **ជាបន្តិចៗ** (gradually) ក្នុងរយៈពេលកំណត់ ជំនួសឲ្យការផ្លាស់ប្តូរភ្លាមៗ។ Transition ដំណើរការបានដោយស្វ័យប្រវត្តិ ពេលដែល state ផ្លាស់ប្តូរ (ដូចជា `:hover`, `:focus`, ឬ JavaScript class change)។

ពីមុន (No Transition)

ផ្លាស់ប្តូរ **ភ្លាម** – user ពិបាក notice

បន្ទាប់ (With Transition)

ផ្លាស់ប្តូរ **ជាបន្តបន្ទាប់** – smooth, professional

Transition ធ្វើឲ្យ UI មានភាព **polish** និង **professional**។ វាជួយ user ឲ្យ *ដឹង* ថា interface ផ្លាស់ប្តូរ ហើយធ្វើឲ្យបទពិសោធន៍ (UX) កាន់តែល្អ។ Transition ផ្សំជាមួយ Transform, Color, Opacity, ឬ Size properties បានលទ្ធផលដ៏ស្រស់ស្អាត។

SYNTAX

២.១. Syntax របស់ Transition

Transition Syntax ងាយស្រួល – មាន Properties ចំនួន ៤ ដែលជំរុញ animation ទាំងមូល។

> `transition: property duration timing-function delay;`

Property	ឧទាហរណ៍	អត្ថន័យ
`transition-property`	`background-color`	CSS property ដែលចង់ animate
`transition-duration`	`0.3s`	រយៈពេល animation
`transition-timing-function`	`ease`	ល្បឿន/curve នៃ animation
`transition-delay`	`0.1s`	ពេលពន្យារ មុនចាប់ផ្តើម

ប្រើ `all` ជា property ដើម្បី animate properties ទាំងអស់ក្នុងពេលតែមួយ ប៉ុន្តែ approach នេះអាចធ្វើឲ្យ performance យ៉ាប់។ ជ្រើសរើស properties ជាក់លាក់ ដើម្បី performance ល្អ។

PROPERTIES

២.២. Transition Properties សំខាន់ៗ

ខាងក្រោមជា Transition Properties ដែលប្រើញឹកញាប់ជាងគេ ក្នុង web development ។

Property	Default	ការប្រើប្រាស់
`transition-property`	`all`	ជ្រើស property ជាក់លាក់ ឬ `all`
`transition-duration`	`0s`	ប្រើ `0.2s-0.5s` សម្រាប់ UI
`transition-timing-function`	`ease`	ease, linear, ease-in, ease-out
`transition-delay`	`0s`	ពន្យារ animation មុនចាប់ផ្តើម
`transition` (shorthand)	–	រួបរួម properties ទាំង ៤ ខាងលើ

```
/* Shorthand Example */
.button {
 transition: background-color 0.3s ease,
 transform 0.2s ease-out;
}
```

TIMING FUNCTIONS

២.២. Timing Functions

Timing Function គ្រប់គ្រង **ល្បឿន** (speed curve) នៃ transition ពេចតំណាក់ (pacing)។ Choice នៃ timing function ប៉ះពាល់ directly ដល់ feel/mood នៃ animation។

Value	Curve	អត្ថន័យ
`ease`	ចាប់លឿន → យឺតចុង	ធម្មតា, natural feel (default)
`linear`	ល្បឿនស្មើពេញ	ល្អសម្រាប់ rotation ឬ continuous animation
`ease-in`	យឺតដំបូង → លឿនចុង	ល្អសម្រាប់ elements ចូល (enter)
`ease-out`	លឿនដំបូង → យឺតចុង	ល្អសម្រាប់ elements ចាកចេញ (exit)
`ease-in-out`	យឺតដំបូង + ចុង	Smooth ល្អបំផុត, polished feel
`cubic-bezier()`	custom curve	Custom timing curve ដែលអ្នករចនាខ្លួនឯង

ANIMATABLE PROPERTIES

២.៣. Properties ដែលអាចប្រើ Transition

មិនមែន CSS properties ទាំងអស់ អាច animate បានទេ។ Properties ដែល animate បានល្អ ត្រូវតែមានតម្លៃ **numeric** ឬ **color** ដែលអាច interpolate (គណនា intermediate values) បាន។

Color Properties

- `color`
- `background-color`
- `border-color`

Size Properties

- `width`
- `height`
- `max-width/height`

Spacing Properties

- `margin`
- `padding`
- `border-width`

Visual Properties

- `opacity`
- `transform`
- `box-shadow`

Performance គន្លឹះ: ប្រើ `opacity` និង `transform` ដើម្បី animate ។ Browser optimize properties ទាំងពីរនេះ ដោយប្រើ GPU ដែលធ្វើឲ្យ animation រលូន (60fps) ។ ជៀសវាង animate `width`, `height`, ឬ layout properties ព្រោះ trigger reflow ។

ឧទាហរណ៍

Transition – ឧទាហរណ៍

ឧទាហរណ៍ខាងក្រោមបង្ហាញ Transition ប្រើលើ **button hover effect** និង **card animation** ។

Button Hover Effect

```
.btn {
  background: #4967E9;
  color: white;
  padding: 12px 24px;
  border: none;
  cursor: pointer;
  transition: background-color 0.3s ease,
              transform 0.2s ease-out,
              box-shadow 0.3s ease;
}

.btn:hover {
  background-color: #3451c7;
  transform: translateY(-2px);
  box-shadow: 0 6px 20px rgba(73,103,233,0.4);
}
```

Card Fade + Slide

```
.card {
  opacity: 0;
  transform: translateY(20px);
  transition: opacity 0.4s ease,
              transform 0.4s ease;
}

.card.visible {
  opacity: 1;
  transform: translateY(0);
}
```

ប្រើ JavaScript ដើម្បីបន្ថែម class `.visible` ពេល scroll ដល់ element

ជំពូក ៥

Media Query

Media Query ក្នុង CSS គឺជាបច្ចេកទេសដែលអនុញ្ញាតឲ្យ website **ប្តូរ style** តាម **ទំហំ screen** ឬ device ដូចជា laptop, tablet, និង phone។ វាជាផ្នែកសំខាន់នៃ **Responsive Web Design** ដែលធ្វើឲ្យ website មើលស្អាតលើគ្រប់ឧបករណ៍ ដោយ CSS ថ្លាក់ (apply) ដោយស្វ័យប្រវត្តិ តាម condition ដែលបានកំណត់។

Mobile First

ចាប់ផ្តើមរចនាសម្រាប់ phone ជាមុន បន្ទាប់មក scale up ទៅ desktop

Adaptive Design

Layout, font size, spacing ផ្លាស់ប្តូរ ស្វ័យប្រវត្តិ តាម screen

Desktop Enhanced

Desktop ទទួលបាន features និង content ច្រើនជាង mobile

SYNTAX

៣.១. Syntax របស់ Media Query

Media Query ប្រើ At-rule `@media` ដើម្បីកំណត់ condition ។ CSS rules ខាងក្នុង `{}` នឹង apply លុះត្រាតែ condition ជា true ។

> `@media media-type and (condition) { CSS rules }`

Media Types

- `all` – គ្រប់ devices ទាំងអស់ (default)
- `screen` – screen/monitor
- `print` – ពេល print
- `speech` – screen readers

Logical Operators

- `and` – conditions ទាំងពីរត្រូវ true
- `or` – conditions ណាមួយ true
- `not` – negate condition
- `,` (comma) – ប្រើជំនួស or

```
/* Basic Media Query */
@media screen and (max-width: 768px) {
 .container {
 flex-direction: column;
 }
}

/* Multiple Conditions */
@media screen and (min-width: 768px) and (max-width: 1024px) {
 /* Tablet styles only */
}
```

CONDITIONS

៣.២. Media Query Conditions ដែលប្រើញឹកញាប់

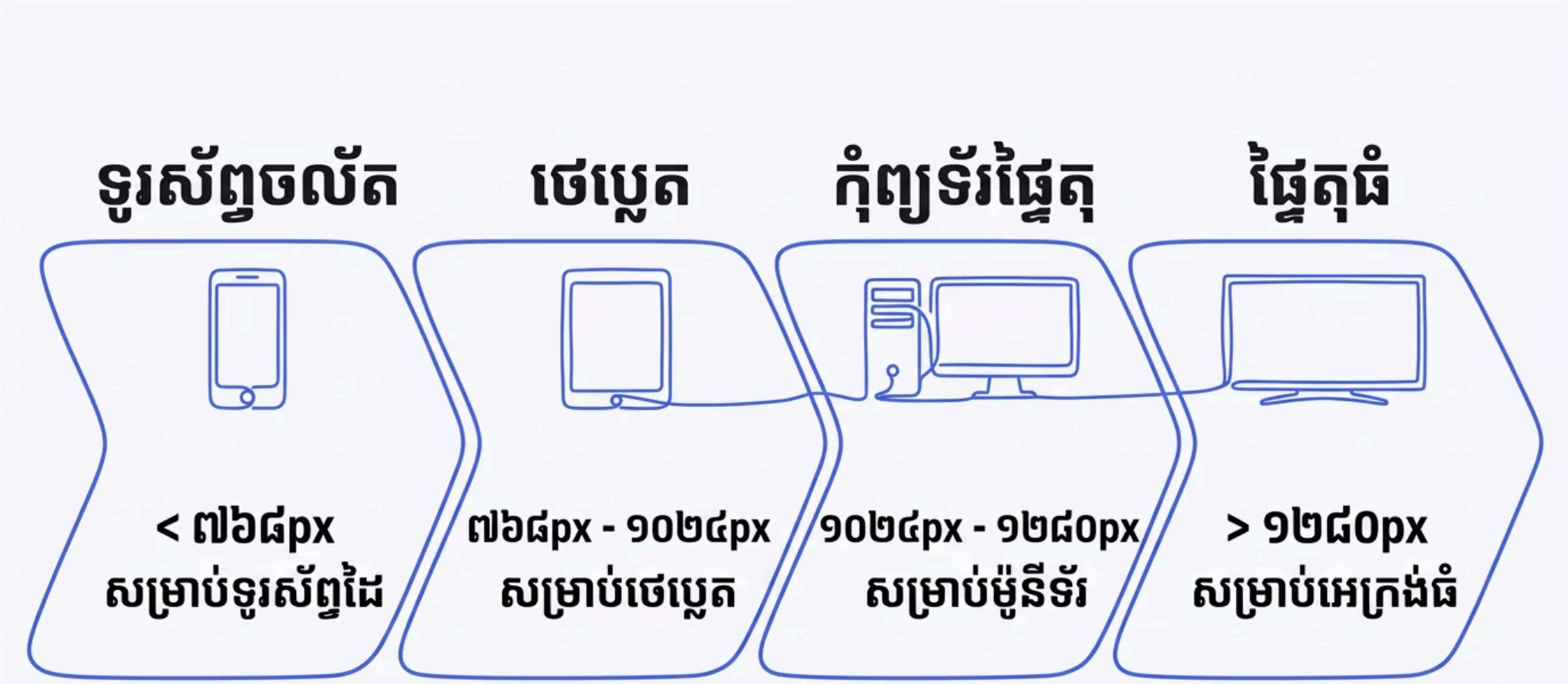
Conditions (ហៅថា **media features**) ជំរុញ logic នៃ Media Query។ ខាងក្រោមជា conditions ដែលប្រើញឹកញាប់ជាងគេ ក្នុង responsive web development។

Condition	ឧទាហរណ៍	អត្ថន័យ
max-width	max-width: 767px	Screen **តូចជាង** ឬ ស្មើ value → apply
min-width	min-width: 768px	Screen **ធំជាង** ឬ ស្មើ value → apply
max-height	max-height: 600px	Height **តូចជាង** ឬ ស្មើ value
min-height	min-height: 400px	Height **ធំជាង** ឬ ស្មើ value
orientation	orientation: portrait	portrait (ឈរ) ឬ landscape (ដេក)
resolution	min-resolution: 2dppx	High-density (Retina) screens
hover	hover: hover	Device ដែលអាច hover (mouse)
prefers-color-scheme	prefers-color-scheme: dark	Dark mode preference

BREAKPOINTS

៣.៣. Responsive Breakpoints

Breakpoints គឺជាចំណុចកំណត់ (thresholds) នៃ screen width ដែល layout ត្រូវប្តូរ។ ខាងក្រោមជា standard breakpoints ដែលប្រើច្រើនជាងគេក្នុង industry។



Device	Width	Media Query
Mobile (Small)	< 576px	`@media (max-width: 575px)`
Mobile (Large)	576px - 767px	`@media (max-width: 767px)`
Tablet	768px - 1023px	`@media (min-width: 768px)`
Desktop	1024px - 1279px	`@media (min-width: 1024px)`
Wide Desktop	> 1280px	`@media (min-width: 1280px)`

អត្ថប្រយោជន៍

៣.៤. អត្ថប្រយោជន៍នៃ Media Query

Media Query គឺជា backbone នៃ **Responsive Web Design**។ ការប្រើ Media Query ជាត្រឹមត្រូវ ផ្តល់អត្ថប្រយោជន៍ច្រើនដល់ project ។

Responsive Website

Layout ផ្លាស់ប្តូរ ស្តាល់ screen size ណាក៏ដោយ ដោយស្វ័យប្រវត្តិ

Mobile Friendly

Google rank ខ្ពស់ជាង ព្រោះ mobile-first indexing ។ UX ល្អ

Layout Adaptive

Layout ប្តូរ columns, font size, spacing ស្រប device

Clean Code

Code ងាយ maintain ជាង ព្រោះ rules ត្រូវបានជ្រើសតែ style ផ្តាច់គ្នា

ឧទាហរណ៍

Media Query – ឧទាហរណ៍

ឧទាហរណ៍ខាងក្រោមបង្ហាញ **Mobile-First Approach** ដែលជា best practice ក្នុង industry – ចាប់ផ្តើម CSS base សម្រាប់ mobile ហើយ progressively enhance សម្រាប់ larger screens។

```
/* === BASE STYLES (Mobile First) === */
.container {
  width: 100%;
  padding: 16px;
}

.grid {
  display: grid;
  grid-template-columns: 1fr; /* 1 column on mobile */
  gap: 16px;
}

.nav {
  display: none; /* Hide nav on mobile */
}

/* === TABLET (768px and up) === */
@media screen and (min-width: 768px) {
  .container {
    max-width: 768px;
    margin: 0 auto;
    padding: 24px;
  }

  .grid {
    grid-template-columns: repeat(2, 1fr); /* 2 columns */
    gap: 24px;
  }

  .nav {
    display: flex; /* Show nav on tablet+ */
  }
}

/* === DESKTOP (1024px and up) === */
@media screen and (min-width: 1024px) {
  .container {
    max-width: 1200px;
    padding: 32px;
  }

  .grid {
    grid-template-columns: repeat(3, 1fr); /* 3 columns */
    gap: 32px;
  }
}
```

Mobile-First គន្លឹះ: ចាប់ផ្តើម CSS base ដោយ **គ្មាន Media Query** (styles ចាស់ apply ដោយស្វ័យប្រវត្តិ) ហើយប្រើ `min-width` Media Query ដើម្បី enhance ទៅ tablet ហើយ desktop។ វិធីនេះ code តូចជាង, load លឿនជាង, ល្អសម្រាប់ SEO និង performance។